

MEDGUARD

Drug–Drug Interaction Detection System

A Multi-Task Clinical NLP System with Knowledge-Graph Enrichment and Conversational AI

Course: SE 4003 — Multidisciplinary Engineering Projects

Instructor: Doç.Dr. Selim Yılmaz

Student: Alaa Hosny Saber Hassouba

Student ID: 220717702

Semester: 2025–2026 Spring

Date: May 2026

GitHub: <https://github.com/Eng-AlaaHosny/MedGuard>

Institution: Muğla Sıtkı Koçman University — Faculty of Engineering, Software Engineering

Abstract

MedGuard is an academic drug–drug interaction (DDI) detection system built on Bio_ClinicalBERT, trained with a three-stage curriculum: Stage 1 recognises drug mentions (NER), Stage 2 classifies interaction type, and Stage 3 estimates clinical severity. The model fuses knowledge-graph embeddings from DrugBank via node2vec (128-d per drug) and Lipinski physicochemical features (5-d per drug) alongside the text encoder, producing a 901-dimensional per-drug representation.

The system is served via FastAPI with a browser demo and an optional Claude-powered conversational assistant.

Metric	Value	Notes
NER macro-F1 (B+I only)	0.9517	Validation set; O class excluded
Interaction-type macro-F1	0.4190	Validation set; 5-class
Severity macro-F1	0.3602	Validation set; 4-class
DDInter coverage	795 / 3,411 (23.31%)	Pairs with resolved severity label

⚠ This system is for academic demonstration only and must not be used for clinical decision-making.

1. Introduction

1.1 The Problem

Every year, more than 1.3 million people in the United States alone are rushed to emergency rooms because of a preventable medication error, at an economic cost exceeding \$3.5 billion annually. Of all medication errors, drug–drug interactions (DDIs) are among the most insidious — they are invisible until they strike.

A DDI occurs when two or more drugs interact in ways that neither would alone: the combination can amplify a drug's effect to toxic levels, cancel it entirely, or produce an unexpected adverse reaction. Warfarin combined with aspirin dramatically increases bleeding risk; amiodarone with digoxin can cause cardiac toxicity; ciprofloxacin with warfarin raises anticoagulant levels through metabolic pathway interference.

The problem is sharpest for the most vulnerable. Elderly patients often manage five to ten medications prescribed by multiple specialists — a cardiologist, endocrinologist, neurologist, and general practitioner — each seeing only their own slice of the patient's medication profile. No single person holds the full picture. Consumer-facing apps generate so many low-priority alerts that clinicians begin to ignore them (alert fatigue). Reference databases like DrugBank are comprehensive but designed for pharmacologists, not patients.

Statistic	Value
Annual US emergency room visits from adverse drug events	1.3 million
Annual economic cost of preventable medication errors	\$3.5 billion+
Typical medication load for elderly patients with chronic conditions	5–10 drugs
Hospital readmissions linked to medication-related problems	~30%

1.2 What MedGuard Solves

MedGuard targets three groups currently underserved by existing tools:

- Patients managing multiple prescriptions — particularly elderly individuals or those with chronic conditions — who need to know whether a new prescription is safe alongside what they already take.
- Caregivers (family members or home-care workers) managing medications on behalf of someone else, who need clear, actionable safety information without requiring pharmacology training.
- Pharmacy students and junior clinicians who benefit from an educational tool that explains not just the interaction result but the reasoning behind it: mechanism, severity rationale, and relevant clinical literature.

MedGuard addresses this by combining four capabilities that no existing consumer tool currently offers together:

- Natural Language Understanding — Users submit two drug names (with optional clinical context text); NER highlights drug mentions in the displayed sentence.
- Risk Stratification — assigns a severity level (safe, caution, warning, danger) trained on DDInter’s curated risk database and expressed in plain colour-coded language.
- Explainability — explains the result through pharmacokinetic or pharmacodynamic mechanism descriptions pulled from DrugBank’s knowledge graph.
- Multi-Modal Intelligence — fuses clinical language (Bio_ClinicalBERT), relational drug knowledge (DrugBank knowledge graph with node2vec embeddings), and molecular chemistry (Lipinski physicochemical features) to combine complementary signals from text, graph structure, and physicochemical descriptors (ablation across single modalities was not performed in this project).

1.3 Scope and Limitations

MedGuard is an academic capstone project developed for SE 4003 at Muğla Sıtkı Koçman University (2025–2026 Spring). It delivers an implemented and evaluated AI core — a trained model, a REST API, and a browser demo — alongside an honest account of its limitations. A full patient-facing React application (with profile management and daily medication scheduler) remains future work; deploying a production patient application on top of an unvalidated model would be irresponsible. The current phase establishes the foundation on which a patient-safe product can be built.

2. Related Work

DDI detection from text is a well-established NLP research problem, anchored by the SemEval-2013 Task 9 shared task and the DDI Corpus it produced. Most prior academic work stops at the NER and classification level. Few systems integrate structural pharmacological knowledge (knowledge graphs) and molecular chemistry (physicochemical features) in a single model.

Fewer still expose their results through an interactive API and a conversational AI interface accessible to non-researchers. MedGuard is designed to bridge that gap.

Reference	Contribution to MedGuard
Bio_ClinicalBERT (Alsentzer et al., 2019)	BERT pre-trained on MIMIC-III clinical notes; used as shared encoder
DDI Corpus / SemEval-2013 Task 9 (Herrero-Zazo et al., 2013)	Sentence-level annotated drug pairs; Stages 1–2 training data
node2vec (Grover & Leskovec, 2016)	Graph embeddings via biased random walks; KG feature extraction
DDInter (Xiong et al., 2022)	Curated drug-pair severity risk levels; Stage 3 training labels
Lipinski Rule of Five (Lipinski et al., 1997)	Five physicochemical drug descriptors; side-feature enrichment
DrugBank 5.0 (Wishart et al., 2018)	Drug names, synonyms, interactions, mechanism text; seeds knowledge graph
Balanced Softmax / Logit Adjustment (Menon et al., 2021)	Long-tail loss function for sparse severity classes

3. System Architecture

MedGuard has two phases: an offline build-and-train phase and an online inference phase. Understanding this split is the key to the whole system — the offline phase produces all the artefacts (knowledge graph, embeddings, checkpoints) that the online phase consumes at request time.

3.1 Offline Phase

The offline phase preprocesses raw data and trains the model. Each script has a single responsibility:

- `preprocessor.py` — parses DDI Corpus XML into `DDISentence` objects.
- `drugbank_processor.py` — builds `drugbank.db` (drugs, interactions, synonyms).
- `kg_builder_full.py` — constructs the NetworkX knowledge graph and computes node2vec embeddings → `knowledge_graph.pkl`.
- `ddinter_vocabulary.py` — loads DDInter CSVs into a normalised name table.
- `trainer.py` — runs the three-stage curriculum pipeline, producing `.pt` checkpoints.

3.2 Online Phase

At startup, `main.py` loads the tokeniser, two model checkpoints, the knowledge graph, and Lipinski data. FastAPI serves two route files:

- `routes.py` — core DDI inference (POST `/api/analyze`).
- `assistant_routes.py` — Claude-powered conversational assistant (POST `/api/assistant/chat`).

The browser demo at `demo.html` calls POST `/api/analyze` and renders colour-coded severity badges.

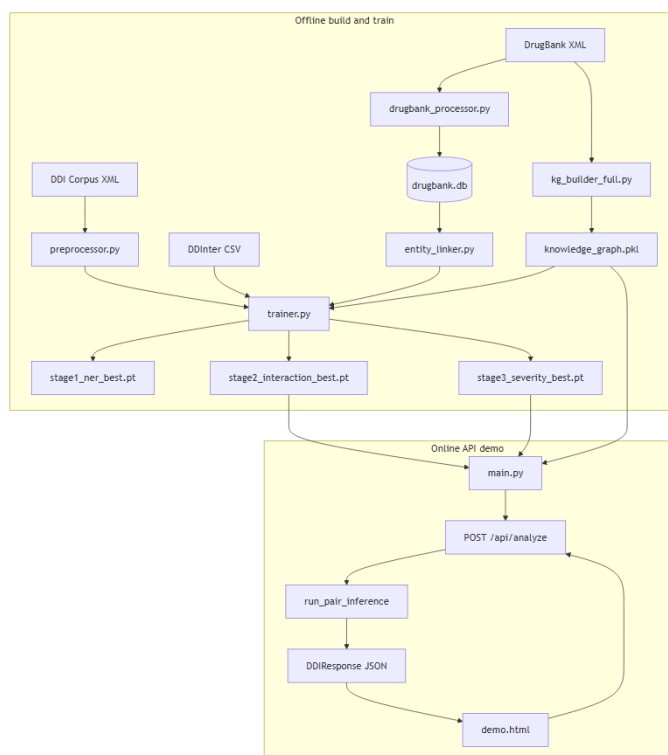


Figure 1. MedGuard system overview: offline data builders and three-stage training produce checkpoints and `knowledge_graph.pkl`; the online phase loads these artefacts in `main.py` and serves POST `/api/analyze` to `demo.html`.

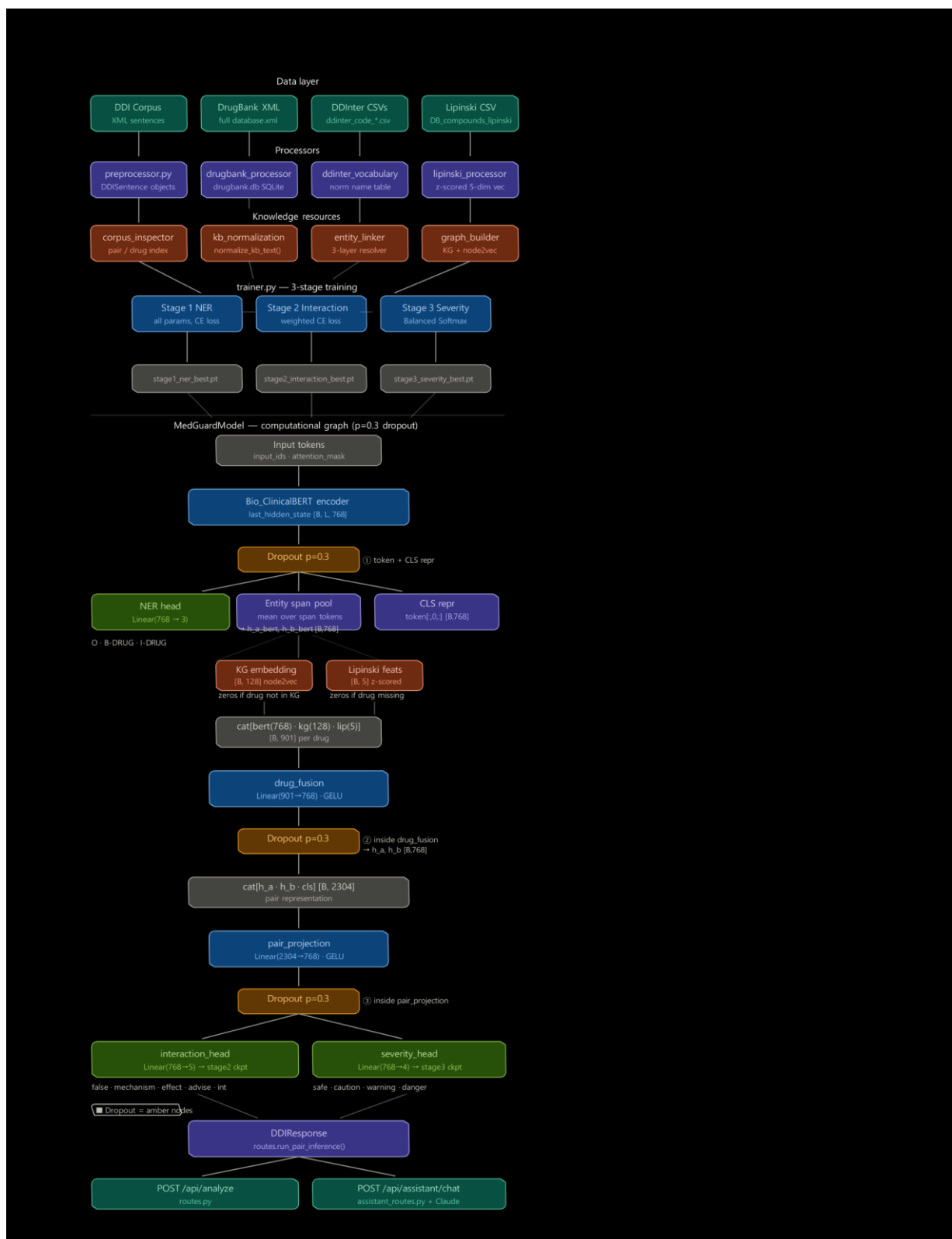


Figure 2. Data layer, processors, and training pipeline: DDI Corpus, DrugBank, DDInter, and Lipinski feed preprocessors and builders that produce drugbank.db, knowledge_graph.pkl, and stage checkpoints consumed at inference.

4. Data Sources & Preprocessing

Before the model can be understood, the data that feeds it must be understood. MedGuard draws on six primary data sources (plus derived artefacts such as `drugbank.db`, `knowledge_graph.pkl`, and training checkpoints), each serving a different role in the pipeline. Text normalisation for all knowledge-base lookups is handled by `normalize_kb_text()` in `kb_normalization.py`: lower-casing, removing parenthetical content, stripping punctuation, and removing salt-form words (hydrochloride, sodium, etc.) so that “Warfarin sodium” and “Warfarin” resolve to the same key.

Dataset / File	Contents	Used For
DDI Corpus (XML)	Clinical sentences, drug entity spans, interaction type labels	Stages 1–2 training; Stage 3 sentences
DrugBank XML	Drug names, synonyms, interactions, mechanism text	Build <code>drugbank.db</code> ; seed knowledge graph
<code>drugbank.db</code> (SQLite)	Drugs, synonyms, interactions, DDInter vocab	Linking, Lipinski ID lookup, API KB context
<code>ddinter_code_*.csv</code>	Curated pairs with risk level (minor / moderate / major)	Stage 3 severity labels
<code>DB_compounds_lipinski.csv</code>	5 physicochemical numbers per compound	Stages 2–3 training; inference side features
<code>knowledge_graph.pkl</code>	NetworkX graph + node2vec embeddings + name maps	128-d drug features; KG context in API
Checkpoints (.pt)	Saved neural network weights per stage	Inference

5. Knowledge Graph Construction

`DrugKnowledgeGraph` (`graph_builder.py`) wraps a NetworkX directed graph where each node is a drug (DrugBank ID) and each edge is a known interaction carrying a severity score (0–3) and description text. The graph is seeded with DDI Corpus drugs; their DrugBank interaction neighbours are added automatically.

`node2vec` is configured with 200 random walks per node of length 30, window 10, and 128-dimensional output (KG_DIM). Drugs with similar interaction profiles are geometrically close in embedding space. At inference, embeddings are retrieved by normalised name; drugs absent from the graph receive a zero vector and `modality_coverage.kg_embedding = false`. When both drugs have a direct edge in the graph, the edge description is returned as `kg_context` in `DDIResponse`, providing human-readable provenance alongside the neural prediction.

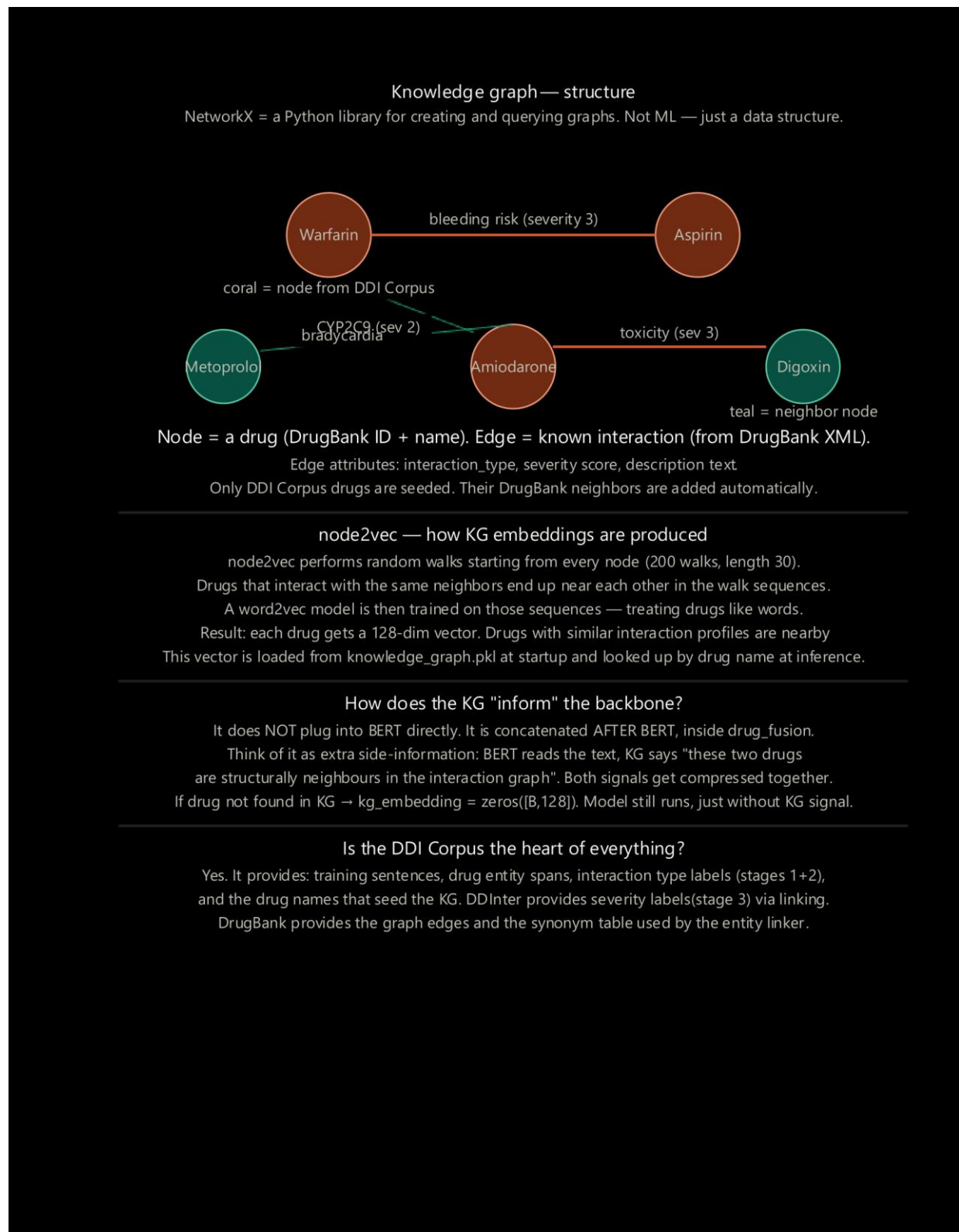


Figure 3. Knowledge graph structure and node2vec embedding pipeline. Nodes are DrugBank drug entities; edges carry interaction severity and description text. node2vec walks produce 128-dimensional embeddings used as side features in the model.

6. Entity Linking

Entity linking is a prerequisite to understanding both the training data and the severity results. DictionaryFirstLinker (entity_linker.py) bridges DDI Corpus drug names to DDInter through a six-step resolver chain. The first matching step wins, and the resolution strategy at each step directly determines which training pairs receive severity labels.

Step	Strategy	Description
1	class_routing	Detects class phrases (e.g. “ACE inhibitor”). Excluded from severity training.
2	ddinter_surface	Normalised name already in ddinter_drug_names table. Direct hit.
3	DrugBank synonym	Synonym table → DrugBank ID → best matching DDInter name.
4	KG maps	Resolved via knowledge_graph.pkl, then cross-referenced to DDInter.
5	drugbank_only	Known in DrugBank but absent from DDInter. Excluded from Stage 3.
6	passthrough	Normalised raw string returned; severity lookup will likely miss.

All resolutions are logged to linking_snapshots.sqlite with a unique run_id (reference run: 854063fe93f2b781). In the reference run: 795 / 3,411 pairs (23.31%) resolved to a DDInter severity label; 539 mentions were skipped via class_routing.

This 23.31% coverage rate is the primary constraint on Stage 3 training diversity and directly explains the lower severity F1 of 0.3602. It is not a modelling failure — it is a data pipeline constraint, rooted in naming mismatches across three incompatible drug vocabularies (DDI Corpus, DrugBank, DDInter). Understanding this is essential context for interpreting the experimental results.

7. Model Architecture

With the data sources, knowledge graph, and entity linking in hand, the model architecture can be understood in full. MedGuardModel (medguard_model.py) uses Bio_ClinicalBERT (768-d hidden) as a shared encoder. The design fuses three distinct signal types per drug into a single learned representation.

7.1 Feature Fusion per Drug

For each drug, three signals are concatenated:

- BERT span mean-pool — 768-d, computed by averaging over the token positions belonging to the drug entity.
- KG node2vec embedding — 128-d per drug, looked up from knowledge_graph.pkl by normalised name.
- Lipinski physicochemical features — 5-d per drug (MW, HBA, HBD, logP, Ro5), z-scored at load time; zeros if drug not found.

Note: During Stages 2–3 training, drug representations use the [CLS] token for efficiency; at inference, explicit token spans from the supplied drug names are mean-pooled (see Section 12.1 for the resulting training/inference mismatch).

The 901-d concatenation passes through drug_fusion: $\text{Linear}(901 \rightarrow 768) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.3)$, yielding a 768-d fused drug vector h_a or h_b .

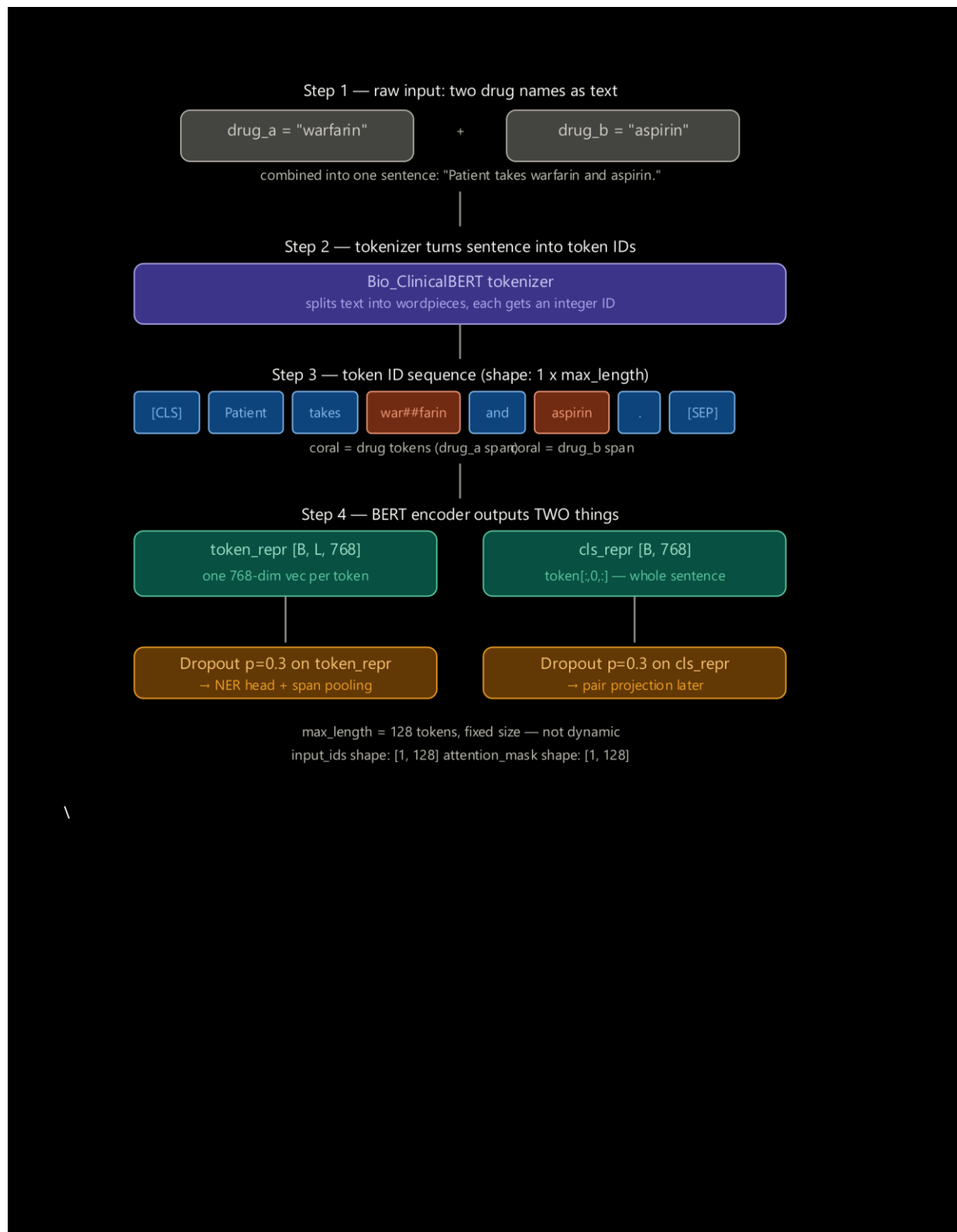


Figure 4. Input representation and backbone flow. Drug names are tokenised by Bio_ClinicalBERT (max 128 tokens); the encoder produces a `token_repr` tensor [B, 128, 768] and a CLS representation [B, 768].

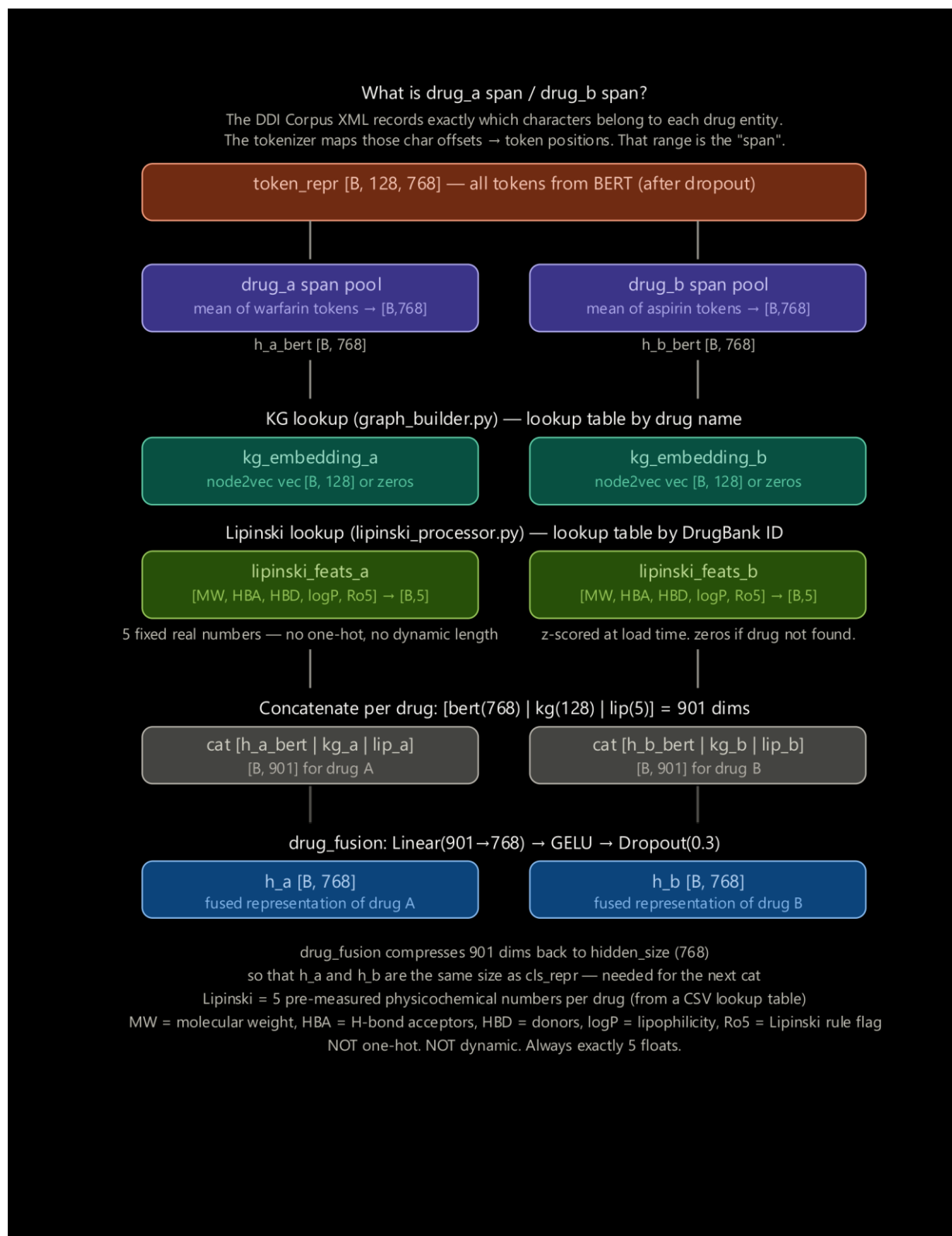


Figure 5. Drug fusion detail. Span pooling extracts `h_a_bert` and `h_b_bert` from `token_repr`; knowledge-graph embeddings (128-d) and Lipinski features (5-d) are concatenated to form the 901-d per-drug vector, then projected to 768-d via the `drug_fusion` MLP.

7.2 Pair Representation and Classification Heads

The pair vector is `cat[h_a | h_b | cls_repr] = 2304-d`, projected through `pair_projection`: `Linear(2304→768) → GELU → Dropout(0.3)`. Two classification heads read this vector:

- `interaction_head` — `Linear(768→5)`, predicting: false / mechanism / effect / advise / int.
- `severity_head` — `Linear(768→4)`, predicting: safe / caution / warning / danger.

A separate NER head reads `token_repr` directly: `Linear(768→3)` for O / B-DRUG / I-DRUG labels.

Component	Input Dim	Output Dim	Checkpoint
Bio_ClinicalBERT encoder	Token IDs (max 128)	768 per token, 768 CLS	shared
NER head	768 (per token)	3 (O/B/I)	stage3
<code>drug_fusion</code> MLP	901 per drug	768 per drug	stage2/3
<code>pair_projection</code> MLP	2304 (768×3)	768	stage2/3
<code>interaction_head</code>	768	5 classes	stage2
<code>severity_head</code>	768	4 classes	stage3

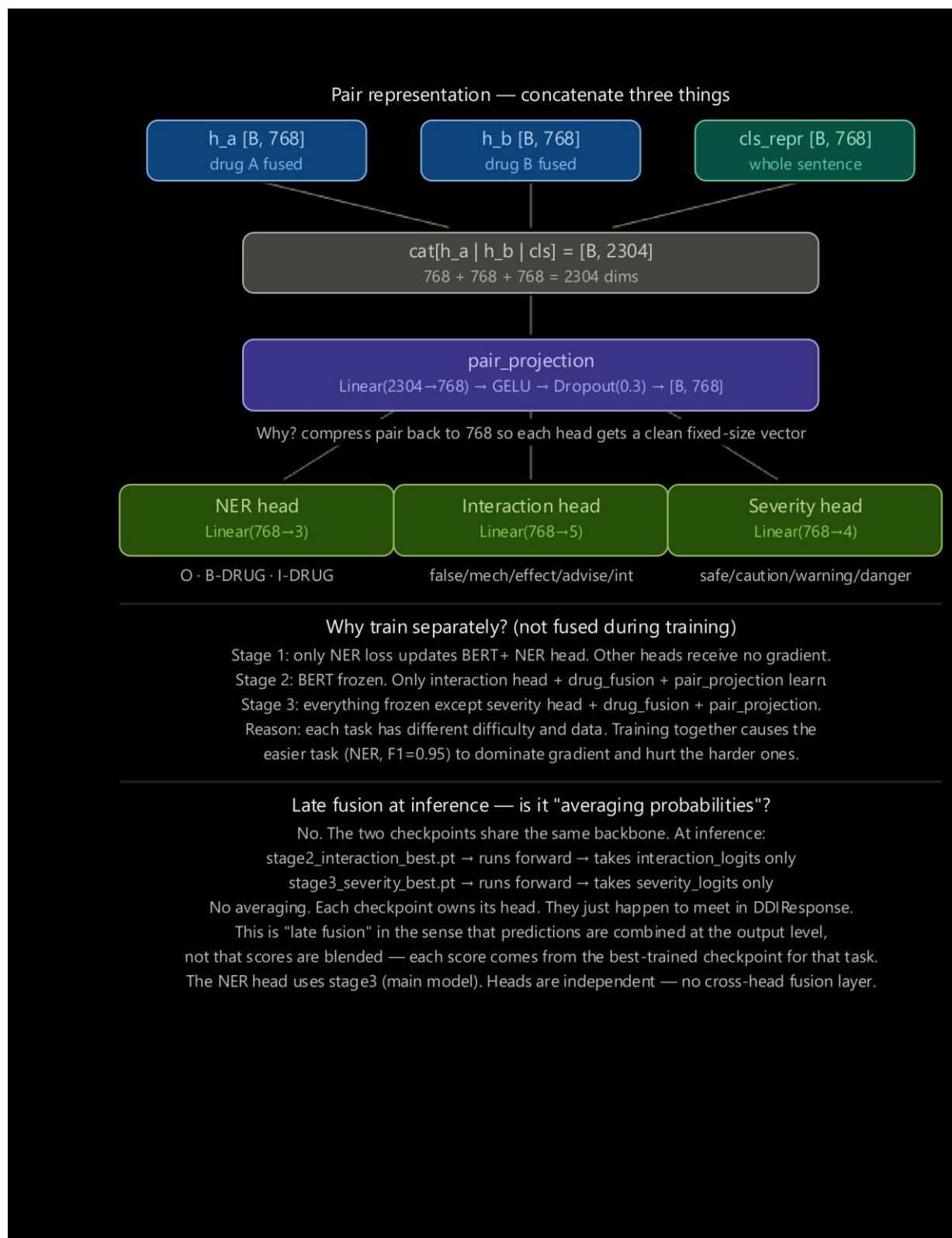


Figure 6. Pair representation, projection, and classification heads. h_a , h_b , and cls_repr are concatenated (2304-d), projected to 768-d, then fed to three independent heads: NER (token-level), interaction_head (5-class), and severity_head (4-class).

7.3 Why Separate Checkpoints?

Each task has different difficulty and data. Training all heads simultaneously lets the easiest task (NER, F1 = 0.95) dominate the gradient and damage the harder ones. The curriculum design ensures each head is optimised independently: Stage 1 trains only BERT and the NER head; Stage 2 freezes BERT and the NER head to train the interaction head; Stage 3 additionally freezes the interaction head to train the severity head.

At inference, two forward passes are run from their respective best checkpoints — there is no score blending. The stage2 checkpoint provides interaction logits; the stage3 checkpoint provides NER and severity logits.

8. Training Procedure

Reproducibility. Trained weights (stage1_ner_best.pt, stage2_interaction_best.pt, stage3_severity_best.pt) are distributed via the project Google Drive folder linked in CHECKPOINTS.md; place them under backend/app/models/checkpoints/ before running main.py.

Run with: `python -m app.models.trainer --stage all`. The three stages execute sequentially, each loading the best checkpoint from the previous stage.

- Stage 1 (NER) — All parameters trainable. Weighted CrossEntropyLoss downweights the O class. Validated on B-DRUG + I-DRUG macro-F1. Saved → stage1_ner_best.pt.
- Stage 2 (Interaction) — Encoder and NER head frozen. Trains drug_fusion, pair_projection, and interaction_head. Balanced class weights address the majority false class. Saved → stage2_interaction_best.pt.
- Stage 3 (Severity) — Interaction head additionally frozen. Trains on DDI Corpus pairs linked to DDInter severity labels via DictionaryFirstLinker. Uses BalancedSoftmaxLoss (logit adjustment) to handle the sparse caution class. Saved → stage3_severity_best.pt.

Hyperparameter	Stage 1 (NER)	Stage 2 (Interaction)	Stage 3 (Severity)
Epochs	5	10	10
Batch size	16	4	4
Learning rate	2e-5	1e-4	1e-4
Gradient accumulation	—	4	4
Loss function	Weighted CE	Weighted CE	Balanced Softmax
Validation metric	Macro-F1 (B+I)	Macro-F1	Macro-F1

9. Inference Pipeline

POST /api/analyze accepts drug_a, drug_b, and optional text. If text is empty, the backend constructs a synthetic sentence: “{drug_a} and {drug_b} may interact. Concurrent use of {drug_a} with {drug_b} should be monitored.”

The text is tokenised by Bio_ClinicalBERT (max_length = 128). find_token_span() locates each drug name’s token indices via offset mapping. KG embeddings and Lipinski features are looked up by drug name or DrugBank ID; missing data yields zero tensors and sets the corresponding modality_coverage flags to false.

Two forward passes are then run — one per checkpoint — and softmax produces probability distributions over class labels. KG context, Lipinski values, plain_guidance, and interaction_reason are assembled into a DDIResponse JSON object.

10. API & Conversational Assistant

10.1 FastAPI Endpoints

Method	Path	Description
POST	/api/analyze	Core DDI inference. Returns DDIResponse JSON.
GET	/api/health	Model, KG, Lipinski, and assistant readiness flags.
GET	/api/drugs	Sample drug names from the loaded knowledge graph.
POST	/api/assistant/chat	Conversational assistant (Claude + tool). Returns reply + tool_trace.

10.2 Conversational Assistant

assistant_routes.py implements an agentic loop using the Anthropic Claude API (default: claude-3-5-haiku-20241022). Claude is given a tool definition medguard_analyze_pair that calls run_pair_inference() — the same function as POST /api/analyze. When Claude emits tool_use blocks, _execute_tool() runs the inference and returns JSON as a tool_result. The loop runs up to MAX_TOOL_ROUNDS = 6.

A system prompt instructs Claude to call the tool for any two-drug query, report severity_label and interaction_type faithfully, and always include an educational-demo disclaimer. Each AssistantChatResponse includes a tool_trace for full transparency.

11. Experimental Results

Evaluation protocol. All reported metrics are macro-F1 on a held-out validation split (15% of sentences). Stage 1 reports macro-F1 on B-DRUG and I-DRUG tokens only (class O excluded). Stages 2 and 3 report macro-F1 over their respective label sets. Results are from a single training run with the hyperparameters listed below; no test-set tuning was performed.

With the full system described, the results can now be interpreted in context. Each metric reflects a specific design decision or data constraint explored in the preceding sections.

Metric	Value	Notes
Stage 1 NER macro-F1 (B+I only)	0.9517	Strong clinical token representations from Bio_ClinicalBERT
Stage 2 interaction-type macro-F1	0.4190	Limited by linguistic subtlety of mechanism/effect/advise/int distinctions
Stage 3 severity macro-F1	0.3602	Constrained by 23.31% DDInter coverage; caution class especially sparse
DDInter coverage	795 / 3,411 (23.31%)	Naming mismatches across three incompatible drug vocabularies
Skipped class-level mentions	539	class_routing exclusions (e.g. “beta blocker”)

Stage 1 NER achieves a strong F1 of 0.9517, reflecting Bio_ClinicalBERT’s pre-training on MIMIC-III clinical notes. Stage 2 interaction-type classification (0.4190) is limited by the linguistic subtlety required to distinguish mechanism, effect, advise, and int classes from each other and from non-interactions. Stage 3 severity (0.3602) is most constrained by data: only 795 of 3,411 sentence pairs carry a resolved DDInter severity label, and the caution class is particularly sparse even within that subset. BalancedSoftmaxLoss mitigates but does not eliminate this imbalance.

DDInter Level	Severity Index	Label
(no match / safe negative)	0	safe
minor	1	caution
moderate	2	warning
major / contraindicated	3	danger

12. Limitations & Future Work

12.1 Current Limitations

- DDInter coverage ~23%: only 795 of 3,411 pairs have severity labels, limiting Stage 3 training diversity.
- Drug-class skipping: 539 class-level mentions (e.g. “beta blocker”) are excluded from severity training — no class-to-ingredient expansion yet.
- Training vs inference span mismatch: Stages 2–3 train with CLS pooling; inference uses explicit span mean-pooling.
- Sentence-level validation split may inflate scores — a pair- or document-level split would be stricter.
- No probability calibration: confidence values are raw softmax, not ECE-corrected.
- No automated test suite (pytest / CI).

12.2 Future Work

- Expand DDInter coverage by incorporating all 26 letter-code CSVs and adding fuzzy name matching.
- Span-consistent training: pass entity spans to forward() in Stage 2/3 datasets to eliminate the training/inference mismatch.
- Apply temperature scaling; report ECE and Brier scores.
- Augment the knowledge graph with KEGG, STITCH, or PharmGKB edges for rare-drug coverage.
- Drug-class-to-ingredient expansion to handle class_routing mentions in severity training.
- Clinical evaluation by pharmacists on a curated drug-pair test set.
- Full patient-facing React application with personal medication profiles and a daily medication scheduler.

13. Conclusion

MedGuard demonstrates that three-stage curriculum learning can effectively train a shared Bio_ClinicalBERT backbone for NER (F1 = 0.95), interaction-type classification (F1 = 0.42), and severity estimation (F1 = 0.36) within a single unified model. The primary bottleneck is DDInter coverage (23.31%), which constrains Stage 3 training data; addressing this through expanded name-matching and additional drug-pair databases is the clearest path to improved severity performance.

The system is fully deployed as a FastAPI REST API with browser demo and Claude-powered conversational assistant. The entity-linking pipeline — resolving naming mismatches across DDI Corpus, DrugBank, and DDInter — is itself a significant engineering contribution. All predictions must be verified by a licensed healthcare professional before any clinical use.

References

Alsentzer, E., et al. (2019). Publicly available clinical BERT embeddings. 2nd Clinical NLP Workshop.

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. ACM SIGKDD.

Herrero-Zazo, M., et al. (2013). The DDI corpus. *Journal of Biomedical Informatics*, 46(5), 914–920.

Lipinski, C.A., et al. (1997). Experimental and computational approaches to solubility/permeability. *Adv. Drug Deliv. Rev.*, 23, 3–25.

Menon, A.K., et al. (2021). Long-tail learning via logit adjustment. ICLR.

Wishart, D.S., et al. (2018). DrugBank 5.0. *Nucleic Acids Research*, 46(D1), D1074–D1082.

Xiong, G., et al. (2022). DDInter: An online drug–drug interaction database. *Nucleic Acids Research*, 50(D1), D1200–D1207.